

Part 2 – Implementation Report Point-of-Sale (POS) System

Group XXX: Member A Member B Member C

January 12, 2026

Contents

1 Introduction

This report documents the implementation phase of our Point-of-Sale (POS) system, developed as the second part of the *Software Engineering Practice* assignment. The project demonstrates the application of Object-Oriented (OO) analysis, design and implementation techniques, supported by modern version-control practices with Git and GitHub.

Core objectives were:

- Provide a working POS application capable of **processing sales** and **handling returns**.¹
- Apply a **three-layer architecture** (UI, Service, Domain) with clear separation of concerns.
- Employ OO principles (abstraction, encapsulation, inheritance, polymorphism) and relevant design patterns (Service Layer, Domain Model).
- Maintain full team collaboration through feature branches, pull-requests and documented Git history.

2 System Overview

2.1 Use Case Model

Figure ?? shows the Use Case Diagram for the POS system. The two primary use cases required by the assignment are **Process Sale** and **Handle Returns**.

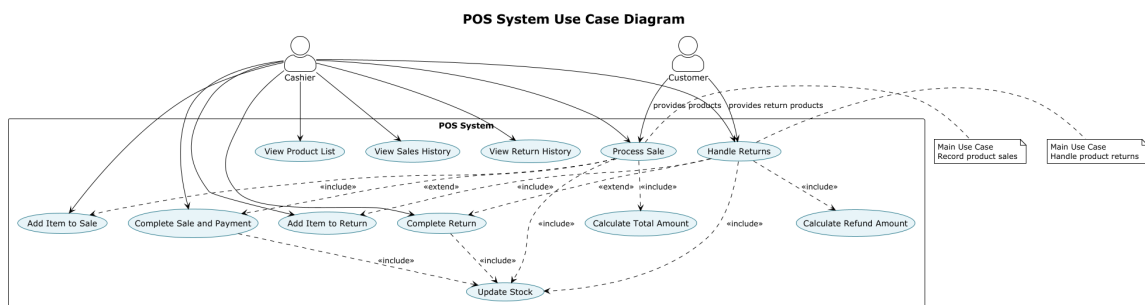


Figure 1: Use Case Diagram of the POS System

2.2 Architecture

The system adopts a three-layer logical architecture. Figure ?? presents the package/-module organisation used in implementation, which helps separate UI, service logic, and domain model concerns.

¹Both CLI and GUI front-ends are available.

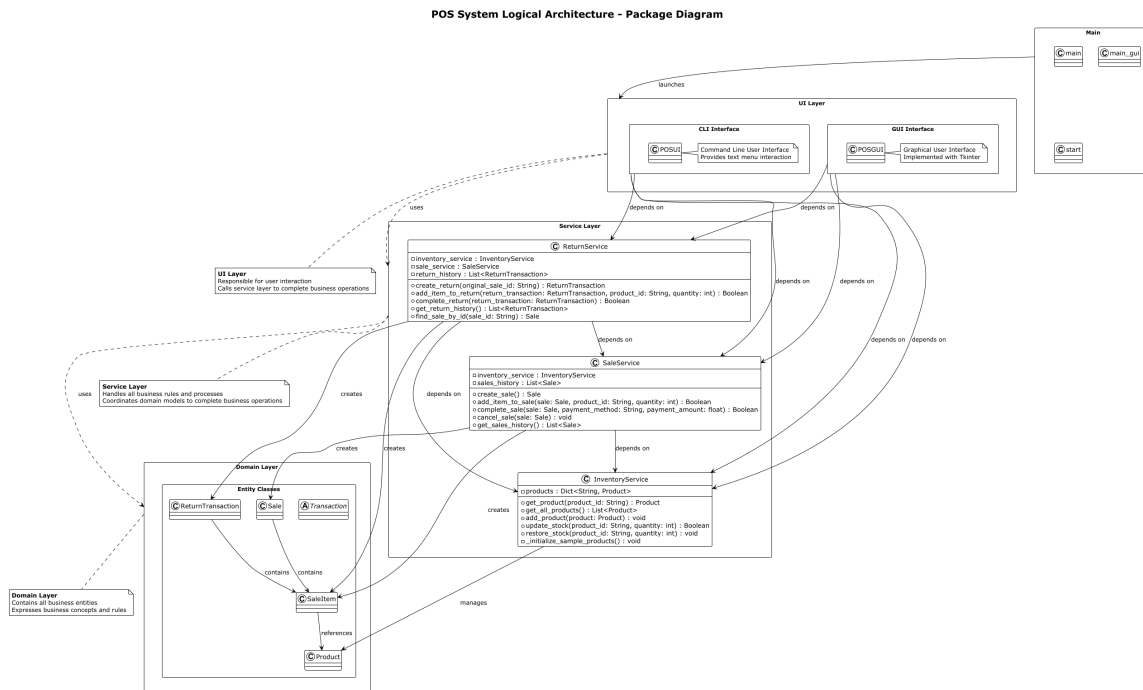


Figure 2: Logical Architecture Package Diagram

2.3 Domain Model

Key business entities appear in the domain layer (Figure ??). They are implemented in the domain/ package and form the backbone of the application logic.

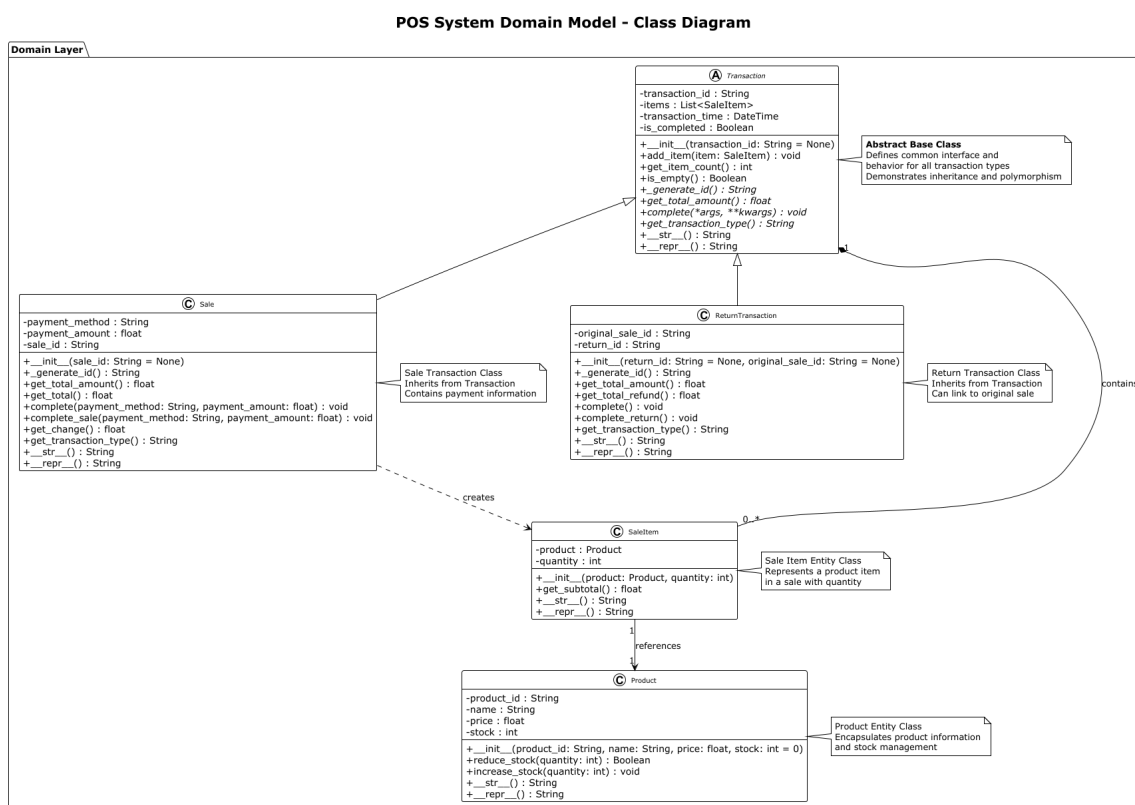


Figure 3: Domain Model Class Diagram

3 System Development

3.1 Tasks Completed

Table 1 summarises the functional and documentation artefacts delivered.

Category	Deliverables
Project Structure	Three-layer package skeleton (<code>domain/</code> , <code>service/</code> , <code>ui/</code>)
Domain Layer	<code>Product</code> , <code>Sale</code> , <code>SaleItem</code> , <code>ReturnTransaction</code>
Service Layer	<code>InventoryService</code> , <code>SaleService</code> , <code>ReturnService</code>
UI Layer	CLI (<code>pos_ui.py</code>), GUI (<code>pos_gui.py</code>) built with Tkinter
Core Use-Cases	Process Sale, Handle Returns (end-to-end)
Persistence	In-memory store; interface prepared for future DB layer
Tests	<code>test_pos_system.py</code> unit / integration tests
UML Artefacts	Use-Case diagram, SSDs, Domain Class diagram, Package diagram, Interaction diagrams (PlantUML)
Documentation	README, ARCHITECTURE, PROJECT_SUMMARY, this Report

Table 1: Summary of completed development tasks

3.2 Interaction Diagrams

To illustrate the dynamic behaviour of key use cases, we include two sequence diagrams generated with PlantUML.

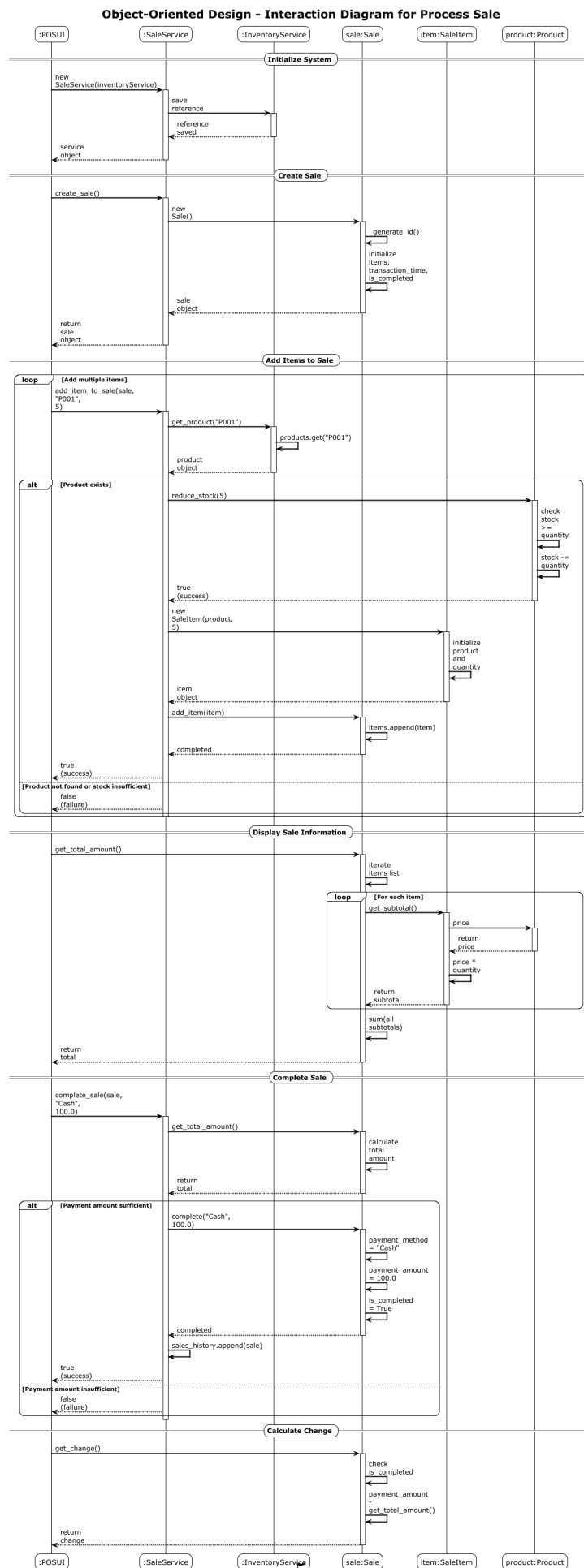


Figure 4: Interaction Diagram – Process Sale

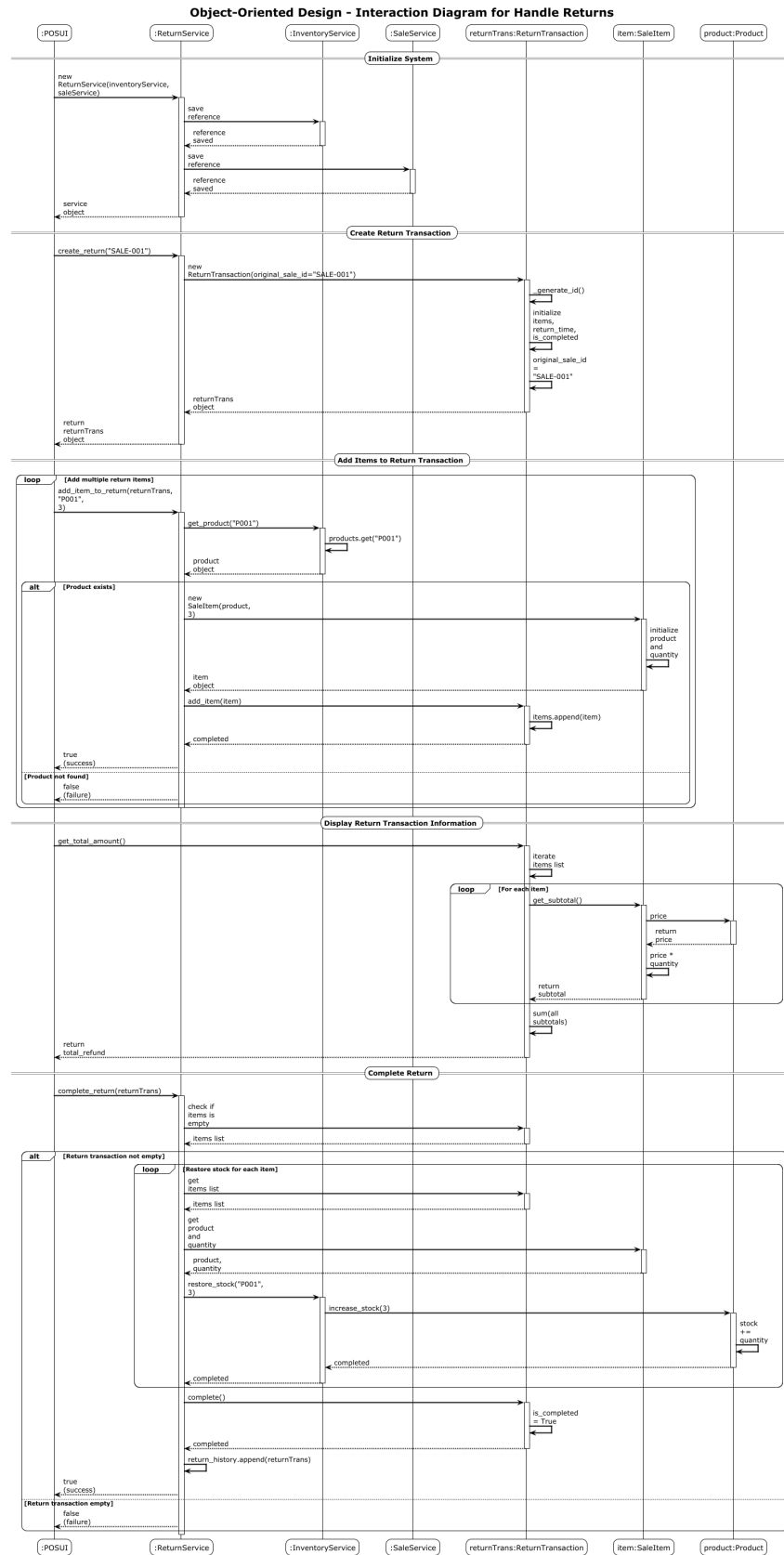


Figure 5: Interaction Diagram – Handle Returns

4 Implementation Snippets

Rather than embedding the entire code-base, representative excerpts are included using `lstinputlisting`. Full source is available in the repository.

4.1 Domain Layer

```

1  """
2  Transaction Abstract Base Class
3
4  """
5
6  from abc import ABC, abstractmethod
7  from datetime import datetime
8  from typing import List
9  from domain.sale_item import SaleItem
10
11
12  class Transaction(ABC):
13      """
14
15
16
17      """
18
19      def __init__(self, transaction_id: str = None):
20          """
21
22
23          Args:
24              transaction_id: IDNone
25          """
26          self.transaction_id = transaction_id or self._generate_id()
27          self.items: List[SaleItem] = []
28          self.transaction_time = datetime.now()
29          self.is_completed = False
30
31      @abstractmethod
32      def _generate_id(self) -> str:
33          """
34              ID
35              ID
36
37          Returns:
38              str: ID
39          """
40          pass
41
42      def add_item(self, item: SaleItem):
43          """
44

```

```
45
46     Args:
47         item:
48         """
49         self.items.append(item)
50
51     @abstractmethod
52     def get_total_amount(self) -> float:
53         """
54
55
56
57     Returns:
58         float:
59         """
60     pass
61
62     @abstractmethod
63     def complete(self, *args, **kwargs):
64         """
65
66
67
68     Args:
69         *args:
70         **kwargs:
71         """
72     pass
73
74     def get_item_count(self) -> int:
75         """
76
77
78     Returns:
79         int:
80         """
81         return len(self.items)
82
83     def is_empty(self) -> bool:
84         """
85
86
87     Returns:
88         bool: True
89         """
90         return len(self.items) == 0
91
92     @abstractmethod
93     def get_transaction_type(self) -> str:
94         """
95
```

```

96         Returns:
97             str:
98             """
99         pass
100
101     def __str__(self):
102         """
103         """
104         items_str = "\n".join(f" - {item}" for item in self.items)
105         status = "Completed" if self.is_completed else "In Progress"
106         return f"{self.get_transaction_type()} {self.transaction_id} ({status})\n{items_str}"
107
108     def __repr__(self):
109         """
110         """
111         return f"{self.__class__.__name__}(transaction_id='{self.transaction_id}', items={len(self.items)})"

```

Listing 1: Transaction Abstract Class

```

1  """
2  Sale Entity Class
3      Transaction
4  """
5
6  from datetime import datetime
7  from domain.sale_item import SaleItem
8  from domain.transaction import Transaction
9
10
11  class Sale(Transaction):
12      """
13
14          Transaction
15
16      """
17
18      def __init__(self, sale_id: str = None):
19          """
20
21          Args:
22              sale_id: IDNone
23              """
24          #
25          super().__init__(sale_id)
26          # Sale specific attributes
27          self.payment_method = None
28

```

```
29     self.payment_amount = 0.0
30
31     # Compatibility aliases
32     self.sale_id = self.transaction_id # old attribute name
33     self.sale_time = self.transaction_time # maintain old
attribute
34
35     def _generate_id(self) -> str:
36         """
37             ID
38
39             Returns:
40                 str: ID
41         """
42         return f"SALE-{datetime.now().strftime('%Y%m%d%H%M%S')}"
43
44     def get_total_amount(self) -> float:
45         """
46
47
48             Returns:
49                 float:
50         """
51         return sum(item.get_subtotal() for item in self.items)
52
53     def get_total(self) -> float:
54         """
55
56
57             Returns:
58                 float:
59         """
60         return self.get_total_amount()
61
62     def complete(self, payment_method: str, payment_amount: float):
63         """
64
65
66             Args:
67                 payment_method:
68                 payment_amount:
69         """
70         self.payment_method = payment_method
71         self.payment_amount = payment_amount
72         self.is_completed = True
73
74     def complete_sale(self, payment_method: str, payment_amount:
float):
75         """
76
```

```

77
78     Args:
79         payment_method:
80         payment_amount:
81     """
82     self.complete(payment_method, payment_amount)
83
84     def get_change(self) -> float:
85         """
86
87
88         Returns:
89             float:
90         """
91         if self.is_completed:
92             return self.payment_amount - self.get_total_amount()
93         return 0.0
94
95     def get_transaction_type(self) -> str:
96         """
97
98
99         Returns:
100             str:
101         """
102         return "Sale"
103
104     def __str__(self):
105         """
106
107         """
108         items_str = "\n".join(f" - {item}" for item in self.items)
109         status = "Completed" if self.is_completed else "In Progress"
110
111         return f"Sale {self.transaction_id} ({status})\n{items_str}\nTotal: ${self.get_total_amount():.2f}"
112
113     def __repr__(self):
114         """
115
116         """
117         return f"Sale(transaction_id='{self.transaction_id}', items={len(self.items)}, total={self.get_total_amount()})"

```

Listing 2: Sale Entity Class

4.2 Service Layer

```

1 """
2 Inventory Management Service
3 """
4
5 from typing import Dict, Optional
6 from domain.product import Product
7

```

```
8
9 class InventoryService:
10     """Inventory management service class"""
11
12     def __init__(self):
13         """Initialize inventory service"""
14         self.products: Dict[str, Product] = {}
15         self._initialize_sample_products()
16
17     def _initialize_sample_products(self):
18         """Initialize sample products"""
19         sample_products = [
20             Product("P001", "Apple", 5.50, 100),
21             Product("P002", "Banana", 3.80, 80),
22             Product("P003", "Milk", 12.00, 50),
23             Product("P004", "Bread", 8.50, 60),
24             Product("P005", "Egg", 15.00, 40),
25         ]
26         for product in sample_products:
27             self.products[product.product_id] = product
28
29     def get_product(self, product_id: str) -> Optional[Product]:
30         """
31         Get product by ID
32
33         Args:
34             product_id: Product ID
35
36         Returns:
37             Product: Product object, or None if not found
38         """
39         return self.products.get(product_id)
40
41     def get_all_products(self) -> list[Product]:
42         """
43         Get all products
44
45         Returns:
46             list: List of all products
47         """
48         return list(self.products.values())
49
50     def add_product(self, product: Product):
51         """
52         Add product
53
54         Args:
55             product: Product object
56         """
57         self.products[product.product_id] = product
58
```

```

59     def update_stock(self, product_id: str, quantity: int) -> bool:
60         """
61         Update stock (decrease)
62
63         Args:
64             product_id: Product ID
65             quantity: Quantity (positive to decrease, negative to
increase)
66
67         Returns:
68             bool: Whether update was successful
69         """
70         product = self.get_product(product_id)
71         if product:
72             if quantity > 0:
73                 return product.reduce_stock(quantity)
74             else:
75                 product.increase_stock(-quantity)
76                 return True
77         return False
78
79     def restore_stock(self, product_id: str, quantity: int):
80         """
81         Restore stock (increase)
82
83         Args:
84             product_id: Product ID
85             quantity: Quantity to restore
86         """
87         product = self.get_product(product_id)
88         if product:
89             product.increase_stock(quantity)

```

Listing 3: Inventory Service

```

1  """
2  Sale Service
3  """
4
5  from typing import List
6  from domain.sale import Sale
7  from domain.sale_item import SaleItem
8  from domain.product import Product
9  from service.inventory_service import InventoryService
10
11
12  class SaleService:
13      """Sale service class"""
14
15      def __init__(self, inventory_service: InventoryService):
16          """
17          Initialize sale service

```

```
18
19     Args:
20         inventory_service: Inventory service object
21     """
22     self.inventory_service = inventory_service
23     self.sales_history: List[Sale] = []
24
25     def create_sale(self) -> Sale:
26         """
27         Create new sale
28
29         Returns:
30             Sale: New sale object
31         """
32         return Sale()
33
34     def add_item_to_sale(self, sale: Sale, product_id: str,
35 quantity: int) -> bool:
36         """
37         Add item to sale
38
39         Args:
40             sale: Sale object
41             product_id: Product ID
42             quantity: Quantity
43
44         Returns:
45             bool: Whether addition was successful
46         """
47         product = self.inventory_service.get_product(product_id)
48         if not product:
49             return False
50
51         if not product.reduce_stock(quantity):
52             return False
53
54         item = SaleItem(product, quantity)
55         sale.add_item(item)
56         return True
57
58     def complete_sale(self, sale: Sale, payment_method: str,
59 payment_amount: float) -> bool:
60         """
61         Complete sale
62
63         Args:
64             sale: Sale object
65             payment_method: Payment method
66             payment_amount: Payment amount
67
68         Returns:
```

```

67         bool: Whether completion was successful
68         """
69         total = sale.get_total()
70         if payment_amount < total:
71             return False
72
73         sale.complete_sale(payment_method, payment_amount)
74         self.sales_history.append(sale)
75         return True
76
77     def cancel_sale(self, sale: Sale):
78         """
79         Cancel sale (restore stock)
80
81         Args:
82             sale: Sale object
83         """
84         for item in sale.items:
85             self.inventory_service.restore_stock(item.product.
product_id, item.quantity)
86
87     def get_sales_history(self) -> List[Sale]:
88         """
89         Get sales history
90
91         Returns:
92             List: List of sales history
93         """
94         return self.sales_history.copy()

```

Listing 4: Sale Service

5 Git Command History

Team collaboration was fully tracked in GitHub. A condensed commit log covering all branches is embedded below (generated on January 12, 2026):

```
% Run in repo root:
```

```
% git log --graph --oneline --all --decorate --since=2025-01-01 > docs/report/git_h
```

6 Screenshots

This section provides an overview of the POS system's graphical user interface. The screenshots highlight the main workflow screens (sales and returns), supporting views (product list), and the transaction history used to verify completed operations.

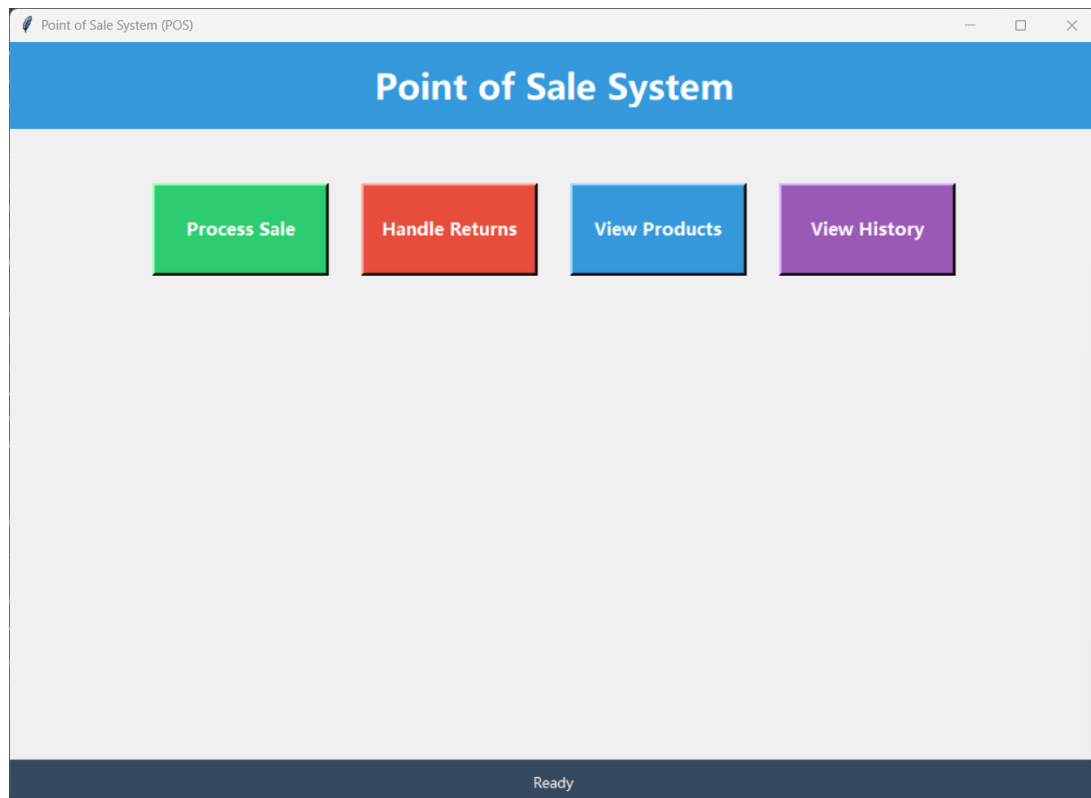


Figure 6: Main Menu

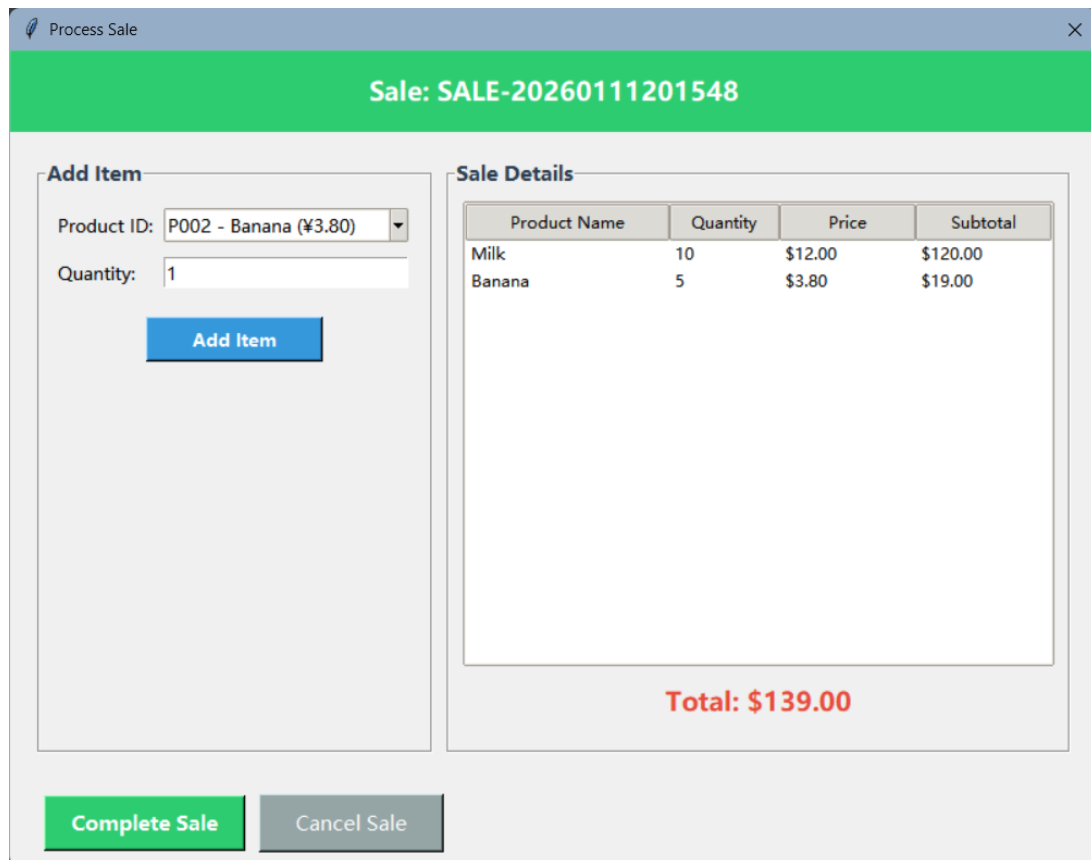


Figure 7: Process Sale Screen

Handle Returns

Return: RET-20260111201925

Add Return Item

Product ID: P001 - Apple (¥5.50)

Return Quantity: 1

Add Return Item

Return Details

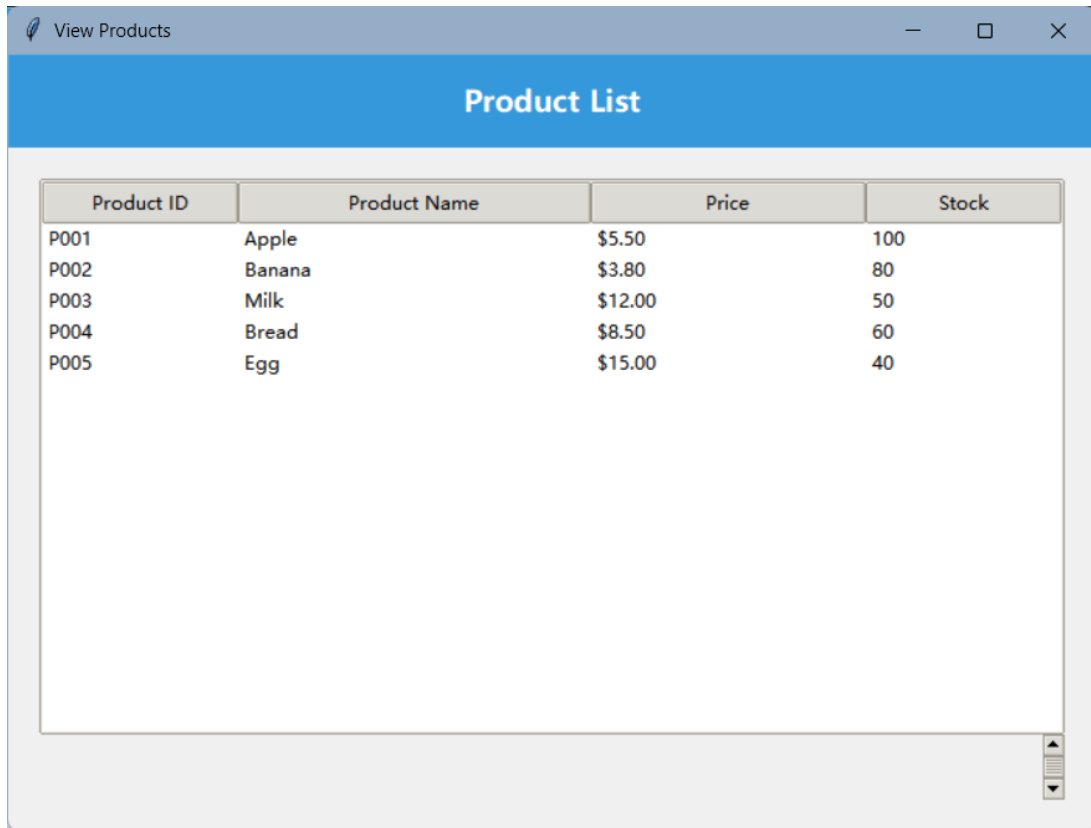
Product Name	Quantity	Price	Subtotal
Apple	10	\$5.50	\$55.00

Total Refund: \$55.00

Complete Return

Cancel Return

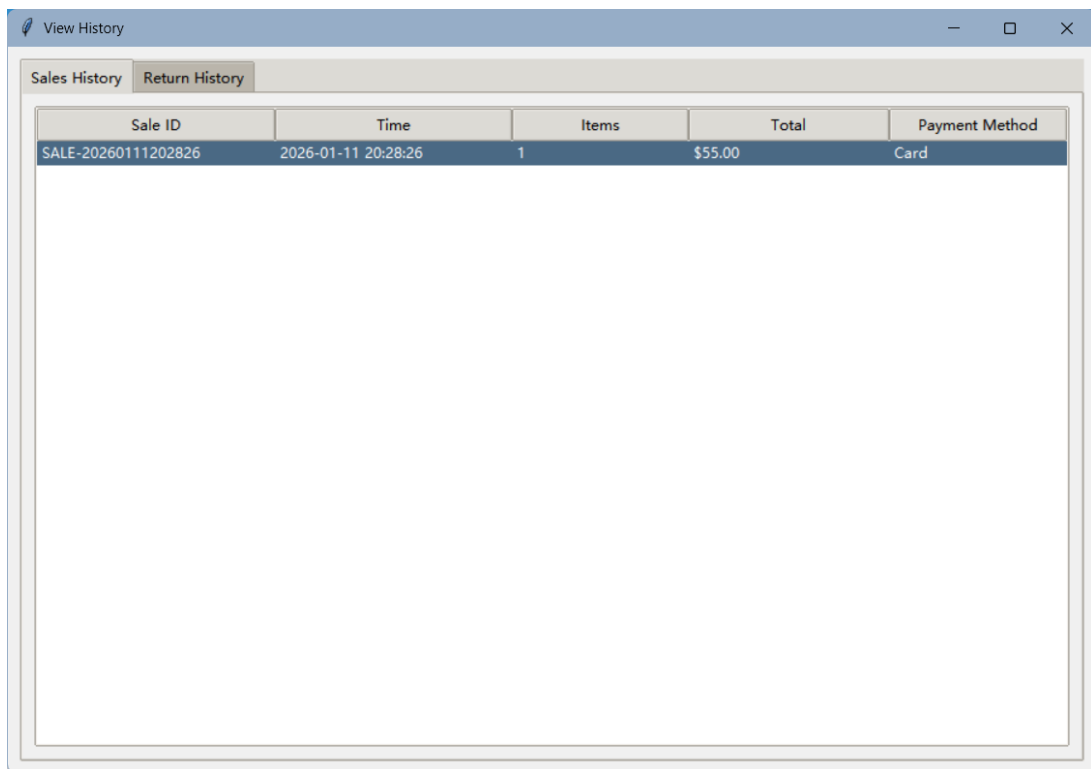
Figure 8: Handle Returns Screen



The screenshot shows a window titled "View Products" with a blue header bar containing the text "Product List". Below the header is a table with four columns: "Product ID", "Product Name", "Price", and "Stock". The table contains five rows of data.

Product ID	Product Name	Price	Stock
P001	Apple	\$5.50	100
P002	Banana	\$3.80	80
P003	Milk	\$12.00	50
P004	Bread	\$8.50	60
P005	Egg	\$15.00	40

Figure 9: Product List Screen



The screenshot shows a window titled "View History" with two tabs: "Sales History" and "Return History". The "Sales History" tab is active. Below the tabs is a table with five columns: "Sale ID", "Time", "Items", "Total", and "Payment Method". The table contains one row of data.

Sale ID	Time	Items	Total	Payment Method
SALE-20260111202826	2026-01-11 20:28:26	1	\$55.00	Card

Figure 10: Sales and Returns History Screen

7 Conclusion

The project successfully delivered a functional POS system demonstrating complete OO lifecycle coverage—from requirements modelling through to working software and documentation. Key take-aways include:

- The three-layer architecture proved effective in separating UI, service logic and domain concerns.
- PlantUML integrated smoothly into the documentation workflow, enabling living UML diagrams.
- Collective code ownership was facilitated by strict branch discipline and frequent merges, resulting in minimal integration conflicts.

Future work may integrate persistent storage, extend reporting capabilities and introduce fine-grained user authentication.